



TigerEye

Cycle 1 Report

Authors:

Amelia Snyder

Mihir Patel

Amy Prieto

Andrea McCarroll

Nicholas McGallagher

Aidan Anderson

Comp 4710 Senior Design Team #28

Department of Computer Science and Software Engineering

Samuel Ginn College of Engineering, Auburn University

Table of Contents

1	System Metaphor (Andrea)	2
2	Cycle Intent (Andrea)	2
3	Management Plan & Project Management (Aidan & Mihir)	3
1.	Semester Assignments	3
2.	Current Assignments	4
3.	Development Schedule.....	5
4.	Meeting Minutes & Cadence	5
4	User Stories (Mihir)	6
5	Design Documentation (Mihir, Nick & Amelia)	7
1.	Overall Product Diagram	7
2.	Parser Engine	7
3.	Alert Engine.....	8
4.	Elasticsearch Database Implementation (Aidan).....	9
5.	Domain Model.....	10
6.	Core Entities (Nouns).....	10
7.	Domain Relationships (Logic)	11
6	Risk Mitigation (Aidan)	13
7	Lessons / Challenges (Amy and Mihir)	14
8	Sponsor Approval (Amy)	14
9	Memoranda (Nick)	15
	January 28, 2026 – TigerEye Meeting Notes	15
	February 11, 2026 – TigerEye Meeting Notes	17
	February 18, 2026 – TigerEye Meeting Notes	18

1 System Metaphor (Andrea)

System metaphor: “A Bird’s Eye in the Sky”

TigerEye can be described to stakeholders as a “Bird’s Eye in the Sky” — a centralized vantage point that rises above individual systems to provide comprehensive awareness of the threat landscape. Like a bird flying high above the ground, TigerEye gives security teams a broad, unobstructed view of their environment instead of limiting them to isolated alerts or disconnected data sources.

From this elevated perspective, TigerEye aggregates network activity and threat intelligence into a single, unified platform. This allows analysts to see patterns, correlations, and anomalies that would be difficult to recognize from a ground-level view. The parser engine acts as the bird’s vision by collecting and translating raw network traffic, threat intelligence provides knowledge of known dangers, and the alert engine serves as the warning signal when suspicious activity is detected.

This higher-level visibility improves situational awareness and enables faster, more confident decision-making. By seeing the full picture, security teams can prioritize threats, detect risks earlier, and respond more effectively. Ultimately, TigerEye transforms fragmented information into clear, actionable insight, allowing defenders to move from reactive responses to proactive security.

2 Cycle Intent (Andrea, Amy)

Cycle takeaways:

In cycle 1 we aimed to build a functional and integrated threat detection pipeline that can monitor network traffic, enrich that data with threat intelligence, and generate meaningful alerts for users. This cycle represents a critical step in building the core functionality of the TigerEye system. Rather than focusing on minor features or interface improvements, the primary goal is to ensure that the system can perform its fundamental purpose: detecting potential threats within network traffic.

One of the key objectives of this cycle was creating our parser engine. We wanted it to successfully ingest network traffic and convert it into structured JSON output. This step is essential because structured data allows other components of the system to process and

analyze the information effectively. Without this capability, the system would not be able to perform threat detection.

Another important objective is integrating threat intelligence from OTX. This involves retrieving IoCs from pulse lists and preparing them for comparison against network data. Threat intelligence provides the knowledge necessary to identify known malicious activity and is essential for making the system useful in real-world environments. The meeting notes emphasize the importance of pulling IoCs and preparing them for comparison by the alert engine.

In addition, this cycle focuses on developing the alert engine so that it can compare parsed network data against threat intelligence and generate alerts when matches are detected. These alerts must be structured in JSON format, so they can be stored, visualized, and analyzed later. This step represents the system's core detection capability.

In cycle 2, we will continue to support the larger objective of building a complete pipeline from network traffic ingestion to structured output. We want to store the results of the parser engine into an elastic search database. Along with that, we aim to develop our OTX knowledge and current groundwork. OTX will be our main focus.

3 Management Plan & Project Management (Aidan, Mihir, Andrea)

1. Semester Assignments

- a. Architect – Aidan Anderson
 - i. Helping with engineering tasks across different project sectors
 - ii. Designing overall application capabilities based on customer's desired MVP
 - iii. Establishing initial data pipelines from McCrary's testbed environment.
 - iv. Integrating our varied sectors with each other to accomplish various objectives and tasks
- b. Backend Engineer - Amelia Snyder
 - i. Developing the initial parser engine to interpret network traffic
 - ii. Researching and implementing best export format to deliver data to our pattern engine/system
 - iii. Formatting IT/OT network traffic, in a uniform manner.
- c. API integration/Backend Engineer - Mihir Patel
 - i. Researching available sources of OSINT related to both IT and ICS environments

- ii. Designing a pattern engine to recognize potential threats based on OSINT sources (API integration)
 - iii. Selecting and implementing database solutions for our overall application
- d. Full Stack Engineer - Nick M.
 - i. Working alongside GUI Engineer to select best visualization tools, languages, and products
 - ii. Ensuring accurate data is integrated into threat cases
 - iii. Tagging data with timestamps to help visualize threat timelines for operators
- e. Graphical Interface Engineer - Andrea McCarroll
 - i. Designing the UX/UI for our operators and analyst in mind
 - ii. Researching pre-built infrastructure that could be leveraged to reduce dev time
 - iii. Creating customized dashboards to present our key features such as event timelines, traffic flow diagrams, and incident case queues
- f. Open-Source Engineer - Amy Prieto
 - i. Providing feedback & instruction on architectural decisions to ensure containerization & open-source capability
 - ii. Managing the GitHub code repository
 - iii. Collaborating in a full-stack capacity, with back end and front-end engineers on key feature capabilities
 - iv. Researching OT cyber domain tools, providing direction on key feature development, and ensuring value to the overall market.

2. Current Assignments

- a. Aidan + Nick - Alert Engine
 - i. Write Python code to generate structured alert files based on Indicators of Compromise (IoCs)
 - ii. Pull pulse data from Open Threat Exchange (OTX) and extract relevant IoCs, primarily IP addresses
 - iii. Compare OTX-derived IoCs against parser-generated or simulated network data
 - iv. Generate structured JSON alert output formatted for future Elasticsearch ingestion
 - v. Establish a functional pipeline from OSINT ingestion to alert generation
- b. Mihir + Amelia - Parser Engine
 - i. Confirm ability to parse DNP3 traffic using the provided test case in the repository

- ii. Ensure parser exports clean, and has structured JSON output
 - iii. Move JSON output into Docker/VM environment running Elasticsearch
 - iv. Validate successful ingestion of parser output into Elasticsearch
- c. Amy + Andrea - Threat Intelligence + UI Concept
- i. Research and compile relevant OTX pulse IDs related to industrial environments and DNP3 traffic
 - ii. Deliver a curated list of usable pulses for the Alert Engine team
 - iii. Develop a GUI mockup for inclusion in the upcoming report
 - iv. Design visual concepts such as alert timeline, case view, and IoC match visualizations
- d. Everyone
- i. Study foundational Threat Intelligence concepts
 - ii. Understand the distinction between IoCs and CVEs
 - iii. Review and become familiar with the Pyramid of Pain to support engine design decisions

3. Development Schedule

- Parser Engine Beta development: February 6th – February 20th
- OSINT API Beta development: February 6th – February 14th
- Task Check-in: February 11th
- Sponsor Check-in: February 19th or 20th (Based on Sponsor's Availability)
- Stretch goal: Live data pipeline established by February 22nd.
- Retrospective & Schedule assessment: March 4th

4. Meeting Minutes & Cadence

- All meetings begin with a retrospective of last week's assigned tasks
- Each member reports on task progression, resource needs, and if they're facing any blockers
- At the end of the meeting, each member tells the team their "sweat level" for the upcoming week, so adjustments can be made based on our joint bandwidth.

Every Wednesday – 6-7 PM	Every Thursday – 6-7 PM
Required weekly meeting to discuss progress and go over tasks to be completed before the next meeting.	Optional workshops to follow up technical obstacles and collaborate on solutions.

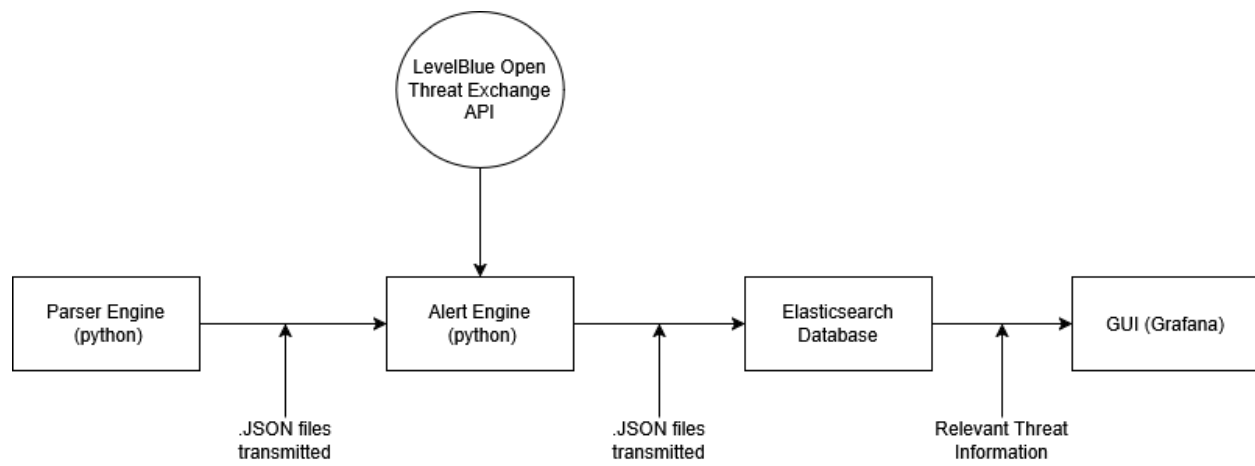
4 User Stories (Mihir)

- As a security operator, I want the ability to view alerted threats, on a specific device, based on open-source intelligence (OSINT), so that I can better assess defensive countermeasures.
- As a SOC analyst, I want unified IT/OT alerts in a single timeline, so that I can identify cross-domain attack patterns.
- As a cyber security professional and system user, I want to have the ability to set up the system to prioritize alerts and events that my company had determined are our Tier 1 level threats, so that I can better customize the system dashboard to fit my needs.
- As a SOC analyst, I want an interactive, real-time baseline of IT/OT behavior with correlated security events, so that I can identify meaningful deviations, understand incident context, and reduce false positives in industrial environments.

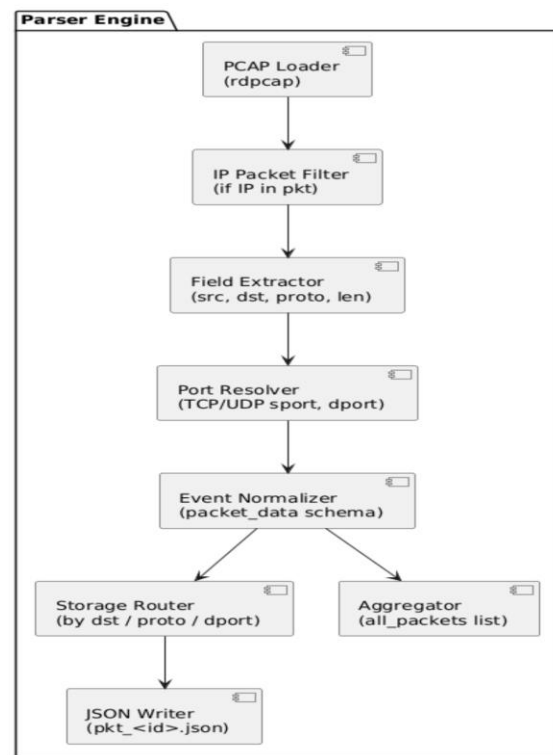
These user stories were collaboratively developed to reflect the core workflow and learning objectives of the TigerEye platform, with each story aligned to support effective threat hunting and situational awareness across IT and OT environments. Together, they emphasize visibility into OSINT-driven device alerts, correlation of IT and OT events within a unified timeline, and the ability to prioritize critical threats based on organizational risk. The final story connects these capabilities through real-time behavioral baselining and interactive visualization, ensuring alerts are contextualized, actionable, and meaningful. Collectively, the user stories define a cohesive workflow that moves from data ingestion and intelligence to correlation, prioritization, and analyst-driven insight. For the Cycle 1 report, our use cases remained the same. However, we thought about each user and how they would use a final product more in depth since our actual development had begun.

5 Design Documentation (Mihir, Nick & Amelia)

1. Overall Product Diagram



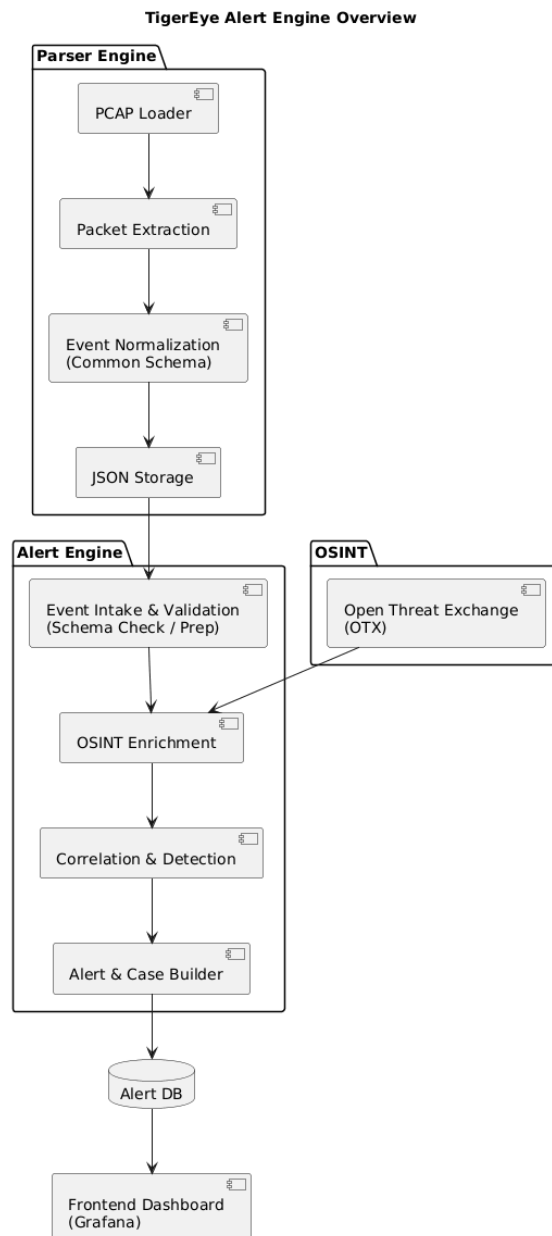
2. Parser Engine



The parser engine's main purpose is to read in .pcap files, sort through their data and collect the relevant information based on our alert engine, save the .pcap logs as .json files, and upload them to a database that can be accessed by the alert engine. To accomplish this, we collected a small sample of .pcap files to test our engine. First, we

loaded the pcap files, then created a basic parse program to read through the files. This program paid attention to several data points from the logs, including the timestamp, the source IP, the destination IP, and the protocols used. It also includes data logs about what the source and destination ports the traffic was using. This information was stored in a dictionary variable, which was later saved in a folder scheme as .json files.

3. Alert Engine



The Alert Engine will begin once structured events are received from the Parser Engine. Its first stage, Event Intake & Validation, will ensure that incoming events conform to the expected schema, verifying required fields while preparing the data for downstream

processing. From there, events will move into OSINT Enrichment, where relevant indicators such as IP addresses, domains, or ports will be queried against external threat intelligence sources, specifically Open Threat Exchange (OTX). In the current development cycle, the primary focus will be retrieving IP address intelligence from OTX and comparing it to the IP addresses extracted by the parser. Any returned intelligence—such as known malicious associations or related threat pulses—will be attached to the event to increase contextual awareness.

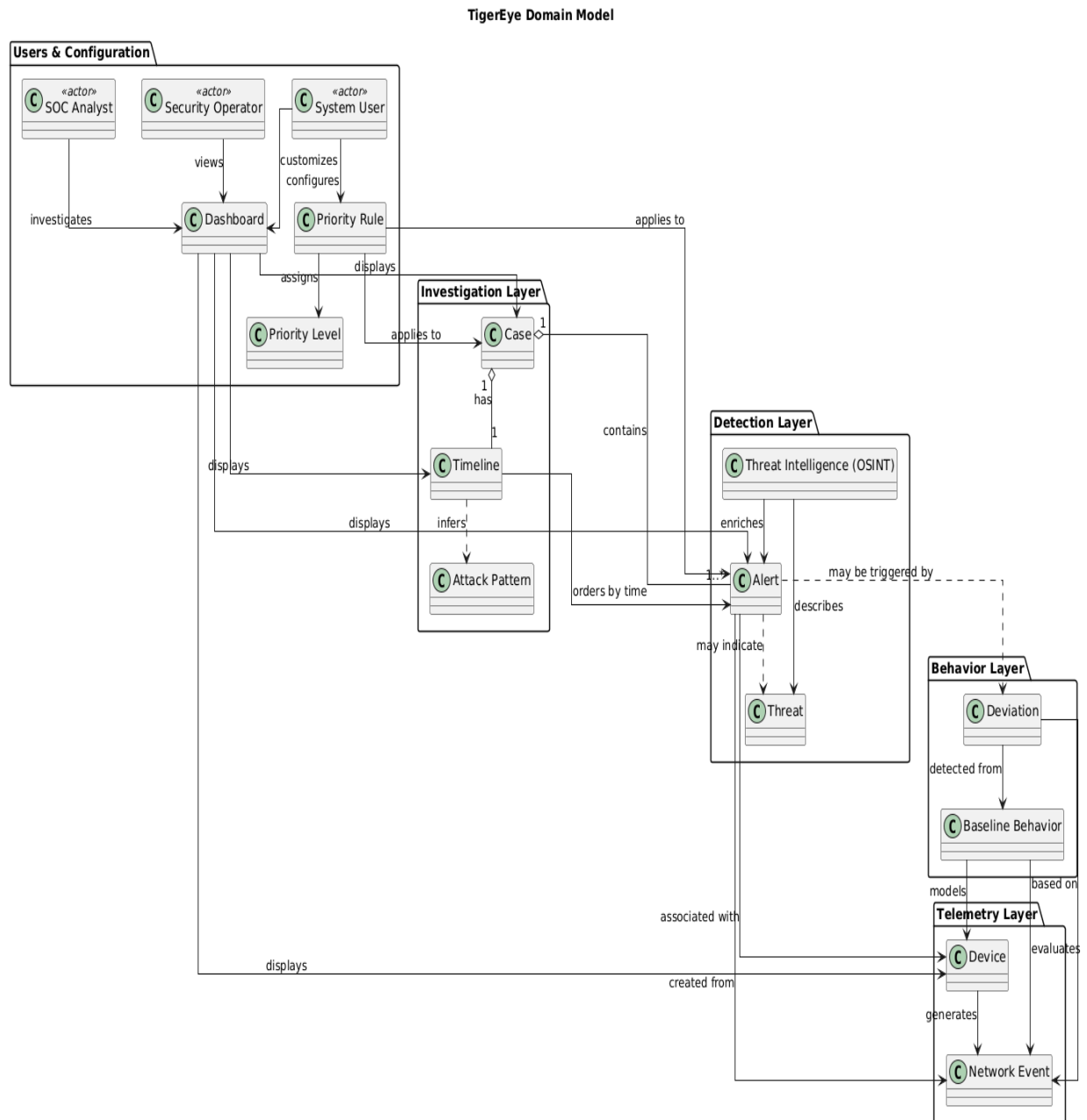
In a later sprint, the now enriched events will pass into a Correlation and Detection phase, where rule-based logic and pattern analysis will be applied to identify suspicious events. This stage will determine whether an event or series of events meets predefined detection criteria. When a detection threshold is met, the Alert & Case Builder will construct a structured alert, assigning a level of severity and confidence. These alerts and cases will then be stored in an Alert Database, where they will be accessible to the frontend dashboard for SOC analysts to review and prioritize.

4. Elasticsearch Database Implementation (Aidan)

Understanding the importance of proper data storage, access control, and scalability, the team agreed to use elastic search database. This critical piece of architecture is used to anchor our data pipeline. All our generated alerts, trivial or critical, are stored here, while also being lightweight. The technical specifications of the device are 6GB of RAM with 100GB of Storage available,

The method we used to install this database we virtualized a Debian 12 Linux VM using VMware Vcenter. Once, we set up the host and placed it on network. We went through the basics of installing Elasticsearch 5.1.2 on to the host. Additionally, we enabled OpenSSH as an available service with controlled keys for designated admins on our team, such as Aidan Anderson and Mihir Patel. We have run some tests of moving our parsed pcap traffic into, but we ran into networking issues. In cycle 2, we will troubleshoot access control standards to repair this issue, but the solution in the interim is using a docker container to store the data locally.

5. Domain Model



6. Core Entities (Nouns)

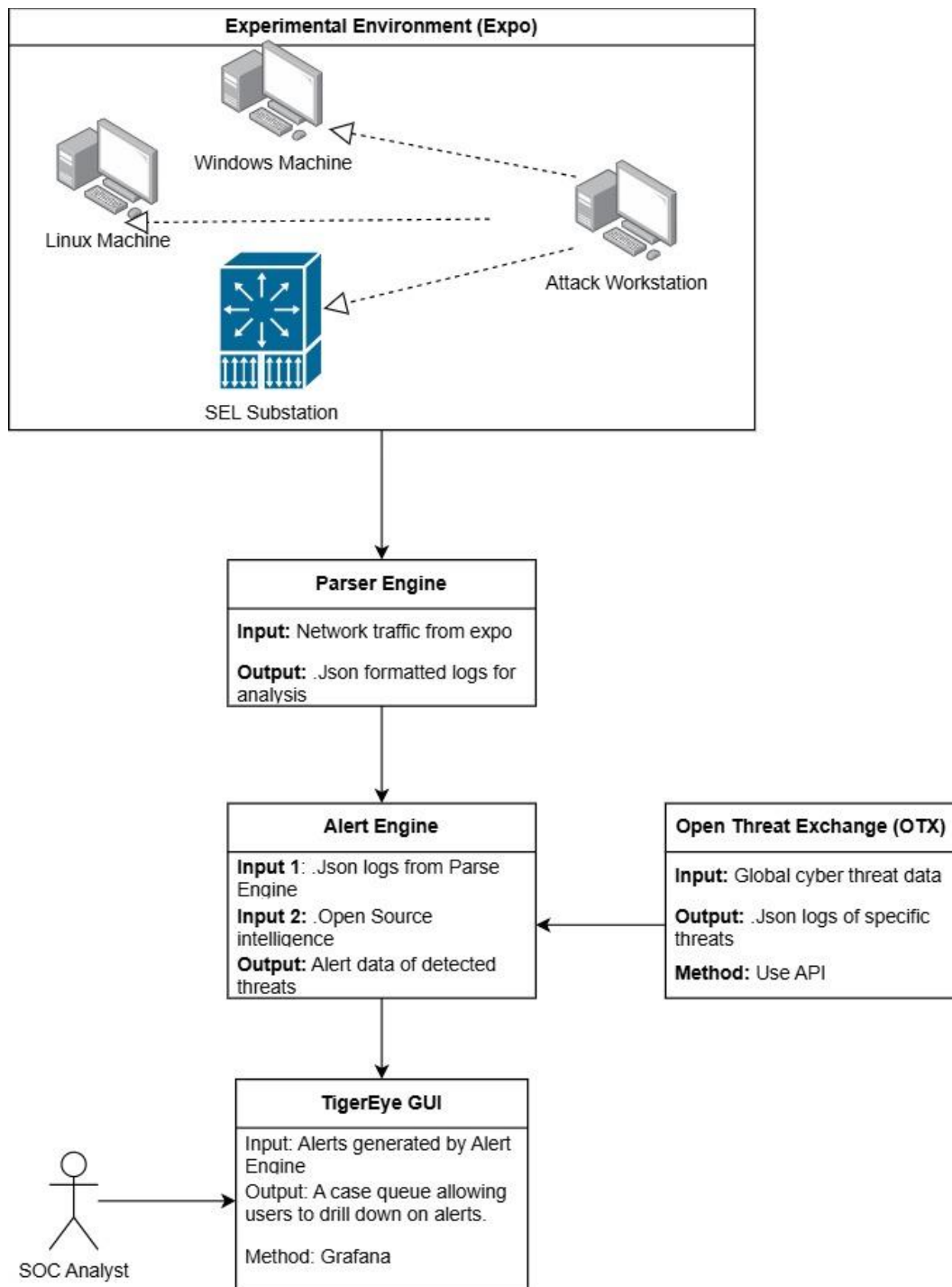
- **Device:** An endpoint (IT/OT) within the Expo environment that generates network events.
- **Network Event:** Raw data (Suricata/SEL logs) captured from a Device.

- **Alert:** A security event triggered when a Network Event matches a known threat pattern.
- **Threat Intelligence (OSINT):** External data from OTX used to "enrich" an Alert with context.
- **Case:** A logical grouping of related Alerts assigned for investigation.
- **Timeline:** A chronological sequence of Network Events and Alerts used to infer attack patterns.
- **Priority Rule:** User-defined logic used to assign a **Priority Level** (e.g., Tier 1) to specific threats.

7. Domain Relationships (Logic)

1. **Telemetry Layer:** A **Device** generates multiple **Network Events**.
2. **Detection Layer:** A **Network Event** is evaluated to create an **Alert**, which is then "enriched" by **Threat Intelligence**.
3. **Behavior Layer:** **Network Events** are measured against **Baseline Behavior** to detect **Deviations**.
4. **Investigation Layer:** Multiple **Alerts** are aggregated into a **Case**, which has a corresponding **Timeline** to help analysts infer an **Attack Pattern**.
5. **Configuration Layer:** A **System User** defines **Priority Rules** that assign **Priority Levels** to specific **Dashboards** and **Cases**.

Conceptual Diagram



6 Risk Mitigation (Aidan)

Risk	Description	Mitigation
Limited tools for interpreting ICS network protocols	Operational technologies tend to have vendor specific net protocols that have limited open-source tools. This could increase our development time overall, due to us having to create specialized classes/functions to parse OT data from scratch.	Our primary mitigation is to use .json as our overall data format. This way we can build a parse engine to take in raw packet capture data, tag interactions with our metadata, and funnel it into our pipeline. Versus, relying on other libraries that might not support industrial protocols.
Lack of reliable OSINT resources with functional APIs	Due to the confidentiality of threats against critical infrastructure, most data sources for traditional artifacts of cyber intelligence (CVEs, IoCs, TTPs) are limited to those with federal connections to CISA. Which creates an intelligence bottle neck for open-source options. Additionally, majority of these threat exchanges do not have federal funding or high commercial investment, so their APIs might not be as refined as mainstream providers such as Mandiant Advantage or Microsoft threat intelligence.	The primary mitigation for under sourced intelligence, is to begin developing your own. From a design perspective, we would shift our focus to verifying data to provide from these sources, instead of using them to inform our platform.

7 Lessons / Challenges (Amy and Mihir)

Lessons : Cycle	Challenges
In Cycle 1 we are learning to navigate new tools like suricata, along with understanding the flow of our team and how we can successfully create our project.	- Understanding best practices to use for our specific mission - Learning new tools, like suricate and Grafana - Researching good open sources for Cyber Intelligence - Coming to a decision on how to format our project
Extended: We have now begun our cycle For the parser engine, we are learning the basic formatting of .pcap and .json files. Between these two formats, we can read in network data logs and create a database that the alert engine can rely on.	Extended: For the parser engines, our challenges were understanding how .pcap data is stored and the best way to extract specific pieces of information, as well as making sure the .pcap files were correctly transformed into .json files.
Extended: Logical Flow Questions For the data flow, the team decided to have the parser engine as the first block, pulling raw network traffic, grabbing the relevant information from each packet, and outputting .json files. Then, the alert engine paired with the OTX API would take the formatted .json files and output relevant threat information (severity, suspicious sources/destinations, unusual amounts of traffic, etc.) in .json files. Those threat .json files would be fed into the Elasticsearch database which would then be called by our GUI for a user to analyze.	Extended: For the logical flow of data, our challenge came from determining what the best way for data to flow through our product was from a development side and an end user side. It was important to determine this when the product is in its infancy as changing the data flow in later cycles would hinder our future development.

8 Sponsor Approval (Amy)

Cycle 1: There have been no new updates from sponsors during this cycle, as they have been away on a work-related trip. However, direction and implementation approvals were previously confirmed in January and early February. Our next team–sponsor meeting is scheduled to occur in March, at which time we will provide progress updates and seek any additional guidance as needed.

9 Memoranda (Nick)

January 28, 2026 – TigerEye Meeting Notes

- Notes
 - EMAIL – Amy sent out an email with our word document for Spike Written Report.
- Role Assignments
 - See Aidan’s text file for exact details (to be uploaded).
 - Unofficial Team Roles
 - Andrea → Slide Maker
 - Amy → Canvas Tracker + Submitter
 - Nick → Note-taker
- Written Report Delegation
 - See Amy’s email for TigerEye Written Report word doc.
 - Deliverable due Feb 4th.
 - Collaboratively...
 - (next meeting) Review, Revise, and Submit on due date February 4, 2026.
 - For this week: work on the report individually Thurs → Fri, then we will see where we are at with it.
 - You might receive a check-in email on Sunday regarding the progress of the report.
- Exit Action Assignments
 - Everyone → Work on the written report, pay attention to email for Azure Cloud invite.
 - Aidan → Looking into Azure and cloud environments.
 - Amelia → Examine data formatting best practices for pcap files, suricata logs, export style for the files, json or others...
 - Recommended it to be json.
 - Mihir → Give ideas on the best database practices, as well as which database structures you’re leaning towards (e.i. Elasticsearch). Let Aidan know if want a virtualized server as well, including any technical specifications relating to OS, etc.
 - Amy → In charge of GitHub. Review containerizations and let know what you should not do to maintain overall efficiency. Jason is available if needed for input on this.

- Nick / Andrea → Research and discuss what we want the frontend to be (e.g. React, Grafana) and be looking for good pre-built visualizations. UI is going to include interactions that give the user the ability to click a case and drill down through alerts, as well as dashboards that will show the timeline of a specific case/cases. Note that these don't need to be integrated yet, that can be something that we do later.
- Wrapping Up
 - If you have run into a blocker, let the group know and we will either figure it out or find another solution. Every time we meet, bring up any blockers (whether it be you don't know something or if you need access to a tool).
 - Tomorrow (1/29/26), Aidan will be working from 6 - 7p, this will be virtual so email if you want to hop on a call and work/discuss.
 - For Nick, make a Meeting Summary channel in Discord that I can upload pdf meeting summaries.

February 11, 2026 – TigerEye Meeting Notes

- This Week – General
 - Aidan
 - Mockup an alert builds for the demo (this is purely status rep related material to have something moving forward).
 - Friday (earliest): Will spin up cloud source / database using specs from Mihir.
 - Get going on the alert engine (python-based backend).
 - Reaffirmed that Grafana will serve as the frontend visualization platform (leveraging its KQL translation capabilities).
 - Has a KQL translator which Amy has experience with.
 - Status Report is Friday morning zoom w/ Dr. Dozier – nothing pressing to ask (if you have something that you want Aidan to submit that morning, ping Aidan on Discord).
- This Week – Coding
 - Use the GitHub (thanks Amy for setting that up) link in the discord.
- Parse Engine
 - Amelia and Mihir are working on this over the week.
 - Work towards getting PCAPs parsed.
 - Read PCAP files.
 - PURELY FUTURE CONCEPTS (credit: Amelia)
 - Live feed to support to automatically reel in PCAPs.

- A middle ground between live feed and self-input would be to allow downloaders to connect their own system for live feeding.
 - Live network taps require “zeek” tool or vm portainer and have everything feed through that.
 - GUI view for Admin in the backend to allow visualization of alerts, etc.
- Alert Engine
 - OTX is the best OSINT so Nick and Aidan will be working on python code for pulling data from OTX and output via a “mythical” JSON file (get closer to this goal at least).
 - Check out modules (this is the time).
 - FAST API is something to investigate.
 - This sprint focuses on OSINT integration and structured data output. Full alert lifecycle management will be implemented in the next sprint.
 - Example of an Alert
 - Malicious IP case: a malicious ip gets sent from the parser, we get the flag on it, send an alert for frontend to present to the user.
- Next Meeting
 - This cycle’s objective is to establish a functional pipeline from network traffic ingestion to OSINT enrichment and structured output.

February 18, 2026 – TigerEye Meeting Notes

- Initial Points
 - We operate in terms of cycles rather than sprints. Cycles avoid the “busy work” that tends to appear near the end of sprints.
- Cycle Reports
 - Good things, good things.
- Upcoming Written Report
 - Deliverable due Feb 18th.
 - For this report, we will basically be building on report 1.
 - FRIDAY – Aidan will check in with Dr. Dozier regarding:
 - See if report can be moved to Wednesday.
 - Figure out what rubric Dr. Dozier is using to grade the report.
 - MONDAY – There will be a team session for working together on the report (6p.).
- Agenda (Battle-Buddy Style)

- alert-engine — Aidan + Nick
 - Write the python to generate a file based on IoCs.
 - Taking in network traffic (from OTX) so that we can compare documented network threats.
 - Deliverables:
 - Pull from pulses from a pulse list, extracting relevant IoCs (primarily IP addresses).
 - Compare IoC against parser-generated network data (or simulated data).
 - Alert JSON formatted for future Elasticsearch ingestion.
- parser-engine — Mihir + Amelia
 - Deliverables:
 - Confirmed ability to parse NDP3 traffic (using the provided test case found in the repo).
 - Parser exports clean and structured JSON output.
 - JSON files successfully moved into Docker/VM running Elasticsearch.
 - Validated ingestion of parser output into Elasticsearch.
- threat intelligence — Amy + Andrea
 - Research into OTX pulse ids related to industrial environments, NDP3 packets, etc. which will be used by the alert-engine later.
 - Deliverables: Need this list of pulses.
 - Andrea – create a mockup for the GUI, that we can attach to the next report so that we can show visually what our tool will look like.
 - Might include alert timeline, case view, IoC match visualizations, etc.
- Homework for Everyone
 - Everyone gets familiar with concepts related to Threat Intelligence (note that IoCs are different from CVEs) and the Pyramid of Pain. This will be crucial for everybody to know to understand the theory behind the engines.